

場域儀表板查詢 API 的規範

基本資訊

項目	內容
實作檔案	api/大概是 API 吧/dashboard/get_family_dashboard.py
方法	POST / CGI stdin
Endpoint	/dashboard/get_family_dashboard.py
Content-Type	application/json; charset=utf-8
App 互動	App 查詢目前場域設備狀態、連線健康度與歷史紀錄。
ESP 互動	無直接 ESP 封包；資料來源為 ESP 狀態回報寫入的 device_telemetry。
稽核	查詢行為寫入 audit_logs，action=DASHBOARD_VIEWED。

POST Request Parameters

欄位	型別	Required	說明
auth_type	string	true	固定 user。
user_id	string	true	查詢者 ID。
family_id	integer	true	目前 App 切換的場域。
include_history	boolean	false	是否回傳歷史 telemetry。
history_limit	integer	false	每台設備最多回傳筆數。

App → API Request Packet

```
{
  "payload": {
    "auth_type": "user",
    "user_id": "admin_001",
    "family_id": 12,
    "include_history": true,
    "history_limit": 5
  }
}
```

Processing Rules

編號	規則
1	驗證 auth_type=user。
2	使用 user_families 確認 user_id 對 family_id 具有 Admin 或 Member 權限。
3	查詢 families、devices。
4	對每台設備查詢最新一筆 device_telemetry 與最新 control_commands。
5	依 recorded_at 與 rssi 判斷 connection_health。
6	若 include_history=true，查詢每台設備最近 history_limit 筆 telemetry。
7	寫入 audit_logs，action=DASHBOARD_VIEWED。

API 對 SQL 寫入內容

資料表	寫入 / 更新欄位	觸發時機與說明
user_families	SELECT role	驗證查詢者是否可看該場域儀表板；Guest 不允許。
devices	SELECT device list WHERE family_id=?	取得場域內設備清單。
device_telemetry	SELECT latest record / history	取得 physical_state、battery、rssi、recorded_at、telemetry_data。
control_commands	SELECT latest command	取得最新控制嘗試與狀態。
audit_logs	INSERT DASHBOARD_VIEWED	記錄使用者查詢儀表板行為。

API → App Response Packet

```
{
  "status": "Success",
  "message": "DASHBOARD_LOADED",
  "data": {
```

```

"family_id": 12,
"viewer": {
  "user_id": "admin_001",
  "role": "Admin"
},
"summary": {
  "total_devices": 1,
  "online_devices": 1,
  "offline_devices": 0,
  "fault_devices": 0,
  "low_battery_devices": 0
},
"devices": [
  {
    "device_id": "ESP32_LOCK_001",
    "physical_state": "UNLOCKED",
    "battery": 87,
    "rssi": -52,
    "connection_health": "GOOD",
    "last_seen_at": "2026-07-07 15:18:55",
    "last_command_id": "CMD..."
  }
],
"history": {
  "included": true,
  "history_limit": 5
}
}
}

```

API ↔ ESP32 封包關係

方向	規範
API → ESP32	本 API 不發控制封包。
ESP32 → API	ESP32 狀態封包先由 device_status_update.py 或 mqtt_status_worker.py 寫入 device_telemetry。
API → App	本 API 將 DB 的最新狀態轉成 App 儀表板 response。

connection_health 判斷

條件	結果
最近 5 分鐘內回報且 RSSI >= -60	GOOD
最近 5 分鐘內回報且 RSSI < -60	WEAK
超過 5 分鐘無狀態回報	OFFLINE
status 為 FAILED / Fault	FAULT
battery <= 20	low_battery=true

Error Responses

HTTP 狀態	錯誤碼	說明
400	INVALID_JSON / MISSING_FIELD	JSON 格式錯誤或缺少必要欄位。
403	ROLE_DENIED / TOKEN_FORMAT_INVALID / TOKEN_EXPIRED / TOKEN_USED_UP	角色不符、訪客令牌格式錯誤、過期或次數耗盡。
404	DEVICE_NOT_FOUND / FAMILY_NOT_FOUND / TOKEN_NOT_FOUND	查無設備、場域或令牌。
409	DEVICE_REVOKED / DUPLICATE_RECORD	設備狀態衝突或不可重複操作。
500	DB_DRIVER_MISSING / INTERNAL_ERROR	資料庫驅動、SQL 或伺服器內部錯誤。

注意事項

- Guest 不允許查詢整個場域儀表板。
- 若設備剛控制成功但未啟動 Worker，mqtt 模式下 Dashboard 可能仍顯示舊狀態。
- App 前端顯示時應區分 last_command 與 physical_state。

App 呼叫範例

App 端以 HTTPS POST 送出 JSON：本機測試可用 curl 或 printf 對應 CGI stdin。

```
POST /dashboard/get_family_dashboard.py
Content-Type: application/json; charset=utf-8
```

```
{
  "payload": {
    "auth_type": "user",
    "user_id": "admin_001",
    "family_id": 12,
    "include_history": true,
    "history_limit": 5
  }
}
```

```
curl -X POST "http://localhost:8000/dashboard/get_family_dashboard.py" \
-H "Content-Type: application/json" \
-d '{"payload": {"auth_type": "user", "user_id": "admin_001", "family_id": 12, "include_history": true, "history_limit": 5}}'
```